University of Dhaka

Department of Computer Science and Engineering

Software Requirements Specification

Academic Operations Suite

Curriculum • Class Routine • Exam Routine
Invigilation Assignment • Thesis Management

CSE 4113- Internet Programming Lab

Submitted By:

Team: Newbie

Marufur Rahman Mithu (19)

Tonmoy Das (22)

Manobendra Biswas (35)

Faisal Ahmed (54)

Course Instructors:

Prof. Dr. Md. Mamun Or Rashid

Professor, Dept. of CSE, University of Dhaka

Md. Fahim Arefin

Lecturer, Dept. of CSE, University of Dhaka

Mr. Md. Ahasanul Alam

Lecturer, Dept. of CSE, University of Dhaka

April 9, 2026

Contents

1 Document Control **3**
1.1 Metadata . . . 3
1.2 Revision History . . . 3

2 Problem Definition **4**
2.1 Problem Statement . . . 4
2.2 Target Users . . . 4
2.3 Goals and Non-Goals . . . 4

3 Scope and Context **6**
3.1 In Scope . . . 6
3.2 Out of Scope . . . 6
3.3 Assumptions and Constraints . . . 6

4 Stakeholders and Roles **8**
4.1 Stakeholders . . . 8
4.2 User Roles and Permissions . . . 8

5 Functional Requirements **9**

6 User Stories and Use Cases **16**
6.1 User Stories . . . 16
6.2 Critical Use Cases . . . 18

7 Data Requirements **21**
7.1 Core Entities . . . 21
7.2 Data Dictionary . . . 22
7.3 Data Model . . . 23

8 UI and UX Requirements **25**
8.1 Wireframes . . . 25
8.2 UX and Accessibility Baseline . . . 26

9 Non-Functional Requirements **27**

10 Risks and Open Questions **29**
10.1 Risk Register . . . 29
10.2 Open Questions . . . 30

11 Testability and Traceability **31**
11.1 Requirement Traceability Matrix 31
11.2 Initial Manual Test Plan 31

Appendix A: MoSCoW Feature Prioritization **36**

Appendix B: API Endpoint Summary **37**

Appendix C: References **39**

1. Document Control

1.1 Metadata

Field	Value
Team	Newbie
Project Name	Academic Operations Suite
Repository URL	https://github.com/manob-endra/AcademicOperationsSuit
Version	0.2
Status	**Draft**
Date	April 8, 2026

1.2 Revision History

Version	Date	Author	Changes
0.1	March 7, 2026	Team Newbie	Initial draft
0.2	April 8, 2026	Team Newbie	Revised Version

2. Problem Definition

2.1 Problem Statement

The Department of Computer Science and Engineering at the University of Dhaka currently relies on manual, spreadsheet-driven processes for managing academic scheduling. This includes building class routines, scheduling examinations, assigning invigilators, tracking thesis supervision, and identifying course gaps for curriculum-switching students. These processes are time-consuming, error-prone, and difficult to communicate to students and faculty.

Scheduling conflicts — double-booked rooms, teacher overlaps, simultaneous exams for the same student section — go undetected until they cause real disruptions. Invigilation duties are distributed unevenly. Students who re-join under a new curriculum cannot easily identify which courses they still need to complete. There is no single authoritative system that students and faculty can consult for up-to-date schedule information.

This matters because scheduling failures directly impact the academic experience of hundreds of students and dozens of faculty members each semester. A centralized, automated solution would reduce administrative overhead, eliminate conflicts, and provide transparent, accessible schedule information to all stakeholders.

2.2 Target Users

User Group	Description	Main Needs
Super Admin	Department chairman or designated officer with full system access	Manage all modules, roles, curricula, rooms
Admin / Officer	Staff managing academic scheduling operations	Create/publish routines, manage exams, invigilation
Teacher / Faculty	Instructors assigned to courses and thesis supervision	View schedules, declare availability, manage thesis
Student	Enrolled students accessing published schedules	View class/exam routines, use missing course finder

2.3 Goals and Non-Goals

Goals

- Automate class and exam routine generation with real-time conflict detection
- Provide multi-version curriculum management with side-by-side comparison
- Fairly distribute invigilation duties respecting teacher availability constraints

- Publish schedules to read-only student and faculty portals
- Identify missing courses for students switching curriculum versions
- Manage thesis supervision assignment and defense scheduling
- Export all schedules and rosters to PDF and Excel formats

Non-Goals

- The system will not handle student grading or academic assessment
- The system will not function as a learning management system (LMS) or course content platform
- Mobile native apps (iOS/Android) are out of scope for this version
- Automated email notifications are a low-priority optional feature, not a core deliverable
- Financial billing features are optional integration only, not a core module

3. Scope and Context

3.1 In Scope

- Google OAuth 2.0 authentication with domain-based role mapping and JWT session management
- Multi-version curriculum creation, editing, comparison, publishing, and archiving
- Class routine generation with room, teacher, and section conflict detection and resolution
- Exam routine scheduling with capacity management and conflict prevention
- Invigilation assignment with workload balancing and teacher constraint support
- Thesis and final project supervision management and defense scheduling
- Missing course finder for students transitioning between curriculum versions
- Published read-only portals for students and faculty with PDF/Excel export
- RESTful API with Swagger/OpenAPI 3.0 documentation

3.2 Out of Scope

- Student grading, transcript management, or GPA calculation
- Learning management features (assignments, lecture notes, video content)
- Native mobile applications — web responsive design only
- Third-party university ERP integration (other than optional billing API)
- Multi-department or multi-university deployment in this version
- Dark mode (deferred to a future version)

3.3 Assumptions and Constraints

Technical Constraints

- Frontend: React.js (v18+), Backend: Supabase — mandated by course
- Database: PostgreSQL — no local file-based storage permitted
- Authentication: Google OAuth 2.0 only — no custom password-based authentication
- Single Page Application (SPA) with RESTful API architecture
- Frontend must be responsive for screen widths 768 px and above

- All deployment on free-tier cloud services (Vercel + Render/Railway or similar)

Organizational Constraints

- Team size is limited to 4 members; feature scope must be managed via MoSCoW prioritization
- Room and course data will be seeded or imported during initial setup by Super Admin

Timeline Constraints

- Project must be completed within a single academic semester
- All Must Have features must be functional before semester-end evaluation

4. Stakeholders and Roles

4.1 Stakeholders

Stakeholder	Interest	Priority
Department Chairman	System governance, academic oversight	High
Admin / Officers	Day-to-day scheduling operations	High
Faculty Members	Routine visibility, invigilation fairness	High
Students	Access to class/exam schedules	High
Team Formula1 (Ext.)	Billing system integration	Low
Course Instructors	Project evaluation and delivery	High

4.2 User Roles and Permissions

Role	Description	Permissions
Super Admin	Full system owner; manages user roles and all modules	Full CRUD on all modules, user role management, system config
Admin / Officer	Manages academic scheduling operations	Create/edit/publish routines, assign invigilation, manage thesis
Teacher / Faculty	Views and responds to assigned duties	View own schedule, declare availability, update thesis status
Student	Read-only consumer of published academic data	View class/exam routines, use missing course finder

5. Functional Requirements

Requirement ID format: `FR-TXX-###`
Priority: **M** = Must Have **S** = Should Have **C** = Could Have **W** = Won't Have

Module 1: Authentication and Role Management

Req ID	Feature	Requirement Statement	Pri.	Acceptance Criteria
`FR-T01-001`	Google OAuth Login	The system shall authenticate users via Google OAuth 2.0, restricting access to `@cs.du.ac.bd` and `@cse.du.ac.bd` domains only.	**M**	Login succeeds with valid DU email; invalid domains rejected with error message
`FR-T01-002`	Domain-Based Role Mapping	Upon first login, `@cs.du.ac.bd` emails assigned Student role; `@cse.du.ac.bd` emails assigned Teacher role.	**M**	Correct role assigned automatically; verified in user management panel
`FR-T01-003`	Super Admin Role Management	Super Admin shall elevate or demote any user role via the User Management panel.	**M**	Role changes take effect immediately; affected user sees updated permissions
`FR-T01-004`	JWT Session Management	System shall issue signed JWT stored in httpOnly cookies, expiring after 24 hours with refresh token support.	**M**	Token expires after 24h; refresh extends session; logout invalidates token
`FR-T01-005`	RBAC Enforcement	All API endpoints and UI routes shall enforce role-based access control; unauthorized access returns HTTP 403.	**M**	Restricted routes return 403 for unauthorized roles; UI hides unauthorized items

Module 2: Curriculum Management

Req ID	Feature	Requirement Statement	Pri.	Acceptance Criteria
FR-T02-001	Create Curriculum	Admins shall create a curriculum specifying program name, year, and course list (code, title, credits, type, semester).	M	New curriculum saved and visible in list; all fields validated
FR-T02-002	Multi-Version Support	System shall maintain multiple independent curriculum versions identified by unique version tags.	M	Two curriculum versions coexist; each independently editable
FR-T02-003	Edit with Versioning	Editing a published curriculum shall auto-create a minor version (e.g., v2023.1) preserving original history.	M	Minor version created on edit; original version unchanged
FR-T02-004	Comparison View	System shall provide side-by-side comparison of two curriculum versions highlighting added, removed, or changed courses.	S	Comparison view shows colored diff between two selected versions
FR-T02-005	Publish / Archive	Admins shall publish (student-visible) or archive (read-only) a curriculum version.	M	Published curriculum appears in student portal; archived cannot be used in scheduling

Module 3: Class Routine Generation

Req ID	Feature	Requirement Statement	Pri.	Acceptance Criteria
FR-T03-001	Create Class Routine	Admins shall create a class routine for a given semester, specifying year, semester, section(s), and courses.	M	Routine created in draft state; all required fields validated before save
FR-T03-002	Assign Teacher to Slot	Admin shall assign a teacher per course slot; system displays current teacher workload for balanced assignment.	M	Teacher assigned; workload counter updates; imbalance flagged with warning
FR-T03-003	Room & Time Slot Mgmt	Admins shall define classrooms (with capacity), labs, and time slots (day, start time, duration).	M	Room created; appears in room selector; capacity enforced

Req ID	Feature	Requirement Statement	Pri.	Acceptance Criteria
FR-T03-004	Real-Time Conflict Detection	System shall detect room, teacher, section, and consecutive-overload conflicts in real time during routine creation.	M	Conflict highlighted immediately when overlapping slot assigned
FR-T03-005	Conflict Report View	A dedicated conflict report shall list all conflicts with details and suggested resolutions.	M	Report lists all conflicts; admin can resolve from this view directly
FR-T03-006	Manual Override (DnD)	Admins shall manually reassign class slots via drag-and-drop; conflicts flagged immediately.	S	Drag-and-drop works; conflict check runs within 1 second of move
FR-T03-007	Publish Class Routine	Admins shall publish a finalized routine, making it visible in student and faculty portals.	M	Status changes to published; routine appears in portals
FR-T03-008	Export Routine	Published routines shall be exportable as formatted PDF and Excel files.	S	PDF and xlsx files download with timetable grid; all slots shown correctly

Module 4: Exam Routine Generation

Req ID	Feature	Requirement Statement	Pri.	Acceptance Criteria
FR-T04-001	Create Exam Schedule	Admins shall create an exam schedule selecting courses, examination period, and available exam rooms.	M	Exam schedule created; all courses assigned to valid slots within period
FR-T04-002	Multi-Sitting Support	Each exam shall support multiple sittings when student count exceeds room capacity.	M	Multi-sitting created automatically when enrollment > room capacity
FR-T04-003	Exam Conflict Detection	System shall detect simultaneous exams for same student group, room over-allocation, and back-to-back warnings.	M	Conflicts blocked before save; back-to-back shows warning allowing override

Req ID	Feature	Requirement Statement	Pri.	Acceptance Criteria
FR-T04-004	Publish & Export Exam Routine	Exam routines shall be publishable and exportable as PDF/Excel; accessible in student portal by year/semester.	M	Exam routine appears in student portal; PDF exports correctly

Module 5: Invigilation Assignment

Req ID	Feature	Requirement Statement	Pri.	Acceptance Criteria
FR-T05-001	Define Invigilation Pool	Admins shall define the eligible invigilator pool per exam session with configurable weights for full-time/part-time faculty.	M	Pool defined; weights stored; pool visible in invigilation module
FR-T05-002	Teacher Constraint Input	Teachers shall declare unavailability; admins can also manually flag unavailability for any slot.	M	Unavailable slots reflected; teacher not auto-assigned to flagged slots
FR-T05-003	Workload-Balanced Assignment	System shall assign invigilators distributing duty count as evenly as possible across the eligible pool.	M	Duty counts are within 1 of the mean across the pool after assignment
FR-T05-004	Conflict Prevention	System shall not assign a teacher who is unavailable, teaching at that time, or already in another room simultaneously.	M	No invigilator assigned to a conflicting slot; all three constraint types enforced
FR-T05-005	Roster Export	A finalized invigilation roster shall be exportable as PDF and Excel; individual teacher views available.	M	Roster PDF/xlsx correct; teacher filter shows only that teacher's duties

Req ID	Feature	Requirement Statement	Pri.	Acceptance Criteria
FR-T05-006	Unassignable Slot Warning & Resolution	When auto-assignment cannot find any eligible invigilator for a slot, the system shall display a persistent red warning banner stating the count of unresolved slots, highlight each affected slot in red with an explanatory tooltip, collect all unresolved slots in a dedicated **Unresolved Slots** section, and block roster publication until every slot is resolved. The admin shall be offered three resolution actions inline: (a) expand the pool, (b) manually override with any faculty member, or (c) navigate to the Exam Routine editor to reschedule the slot.	**M**	Auto-assign with fully constrained pool: red banner appears with correct unresolved count; affected slots highlighted red; Publish button disabled with explanatory tooltip; all three resolution paths functional and tested

Module 6: Thesis Supervision and Defense Scheduling

Req ID	Feature	Requirement Statement	Pri.	Acceptance Criteria
FR-T06-001	Thesis Registration	Admins or teachers shall register a student thesis with student ID, title, and supervisor.	**S**	Thesis record created and linked to student and supervisor
FR-T06-002	Supervisor Assignment	Each thesis assigned a primary and optional co-supervisor; supervisor workload displayed for balanced assignment.	**S**	Supervisee count shown; system warns if count exceeds threshold
FR-T06-003	Defense Scheduling	Admins shall schedule defense events; conflicts with class/exam routines shall be detected.	**S**	Defense scheduled; system warns on room/time conflicts with existing slots

Module 7: Missing Course Finder

Req ID	Feature	Requirement Statement	Pri.	Acceptance Criteria
FR-T07-001	Student Profile Input	Students (or admins) shall input student ID, original curriculum version, and list of completed courses and grades.	S	Profile accepted; completed courses stored and linked to specified curriculum version
FR-T07-002	Equivalency Mapping	Admins shall define course equivalency mappings between curriculum versions used by the missing course finder.	S	Mapping defined; used by finder; unmapped courses flagged
FR-T07-003	Missing Course Calculation	Given completed courses and target curriculum, system computes completed mappings, missing courses, and remaining credits.	S	Calculation correct compared to manual check; missing list and credit totals displayed
FR-T07-004	Export Report	Missing course analysis shall be exportable as a formatted PDF for official use.	S	PDF report includes student info, mappings, missing list, and credit summary

Module 8: Published Portals

Req ID	Feature	Requirement Statement	Pri.	Acceptance Criteria
FR-T08-001	Student Routine Portal	Students shall view published class routines filtered by year, semester, and section in grid and list views.	M	Correct routine shown for student's year/semester/section; both views functional
FR-T08-002	Exam Routine Portal	Students shall view published exam routine for current semester showing date, time, course, and room.	M	Correct exam schedule shown; all fields present
FR-T08-003	Faculty Dashboard	Teachers shall view their own class and invigilation schedule in a personal dashboard filtered by semester.	M	Dashboard shows only teacher's own slots; semester filter works

Req ID	Feature	Requirement Statement	Pri.	Acceptance Criteria
FR-T08-004	Printable PDF View	Both student and faculty portals shall include a Print/Download PDF button.	**S**	PDF download produces a clean, correctly formatted timetable page

6. User Stories and Use Cases

6.1 User Stories

Story ID	As a...	I want...	So that...	Acceptance Criteria
`US-T01-001`	Super Admin	elevate a Teacher account to Admin role	Admin can access scheduling features without Super Admin intervention	Role updated; new Admin can create routines; confirmation shown
`US-T01-002`	Student	log in with my `@cs.du.ac.bd` Google account	I can securely access my personalized schedule without a separate password	Login succeeds with valid DU email; invalid domains rejected; student dashboard visible
`US-T02-001`	Admin	create a new curriculum version (e.g., Curriculum 2023)	the department can adopt an updated course structure without removing the old one	New curriculum saved; all courses listed; version visible with status 'draft'
`US-T02-002`	Admin	compare Curriculum 2016 and 2023 side by side	I can identify which courses were added, removed, or modified for student advising	Comparison highlights new courses (green), removed (red), modified (yellow)
`US-T03-001`	Admin	automatically detect teacher and room conflicts while building the class routine	I can resolve conflicts before publishing and avoid disruptions	Conflict indicator shown in real time; conflict report updated immediately
`US-T03-002`	Admin	drag and drop a class slot to a different time	I can quickly adjust the timetable without re-entering all details	Slot moves; conflict check runs; conflict highlighted if created
`US-T03-003`	Teacher	view my weekly class schedule for the current semester	I can plan my time and attend all assigned classes	Dashboard shows only my own slots with correct day, time, room, course

Story ID	As a...	I want...	So that...	Acceptance Criteria
US-T04-001	Admin	generate an exam routine avoiding simultaneous exams for the same section	students are not forced to sit two exams simultaneously	No two exams for same section overlap; system blocks save on conflict
US-T04-002	Student	view the published exam routine filtered by my semester and section	I know when, where, and what course each exam is for	Exam list shows correct dates, times, course names, and rooms
US-T05-001	Teacher	declare my unavailability for specific exam dates and slots	I am not assigned invigilation duties at times I cannot attend	Unavailable slots saved; not assigned in auto-assignment; admin sees constraint
US-T05-002	Admin	auto-assign invigilators with workload balancing	invigilation duties are distributed fairly across faculty	Each teacher's duty count is within 1 of the average; all constraints respected
US-T05-003	Teacher	view a personalized invigilation roster for the upcoming exam session	I can prepare without searching the faculty-wide roster	Personal roster shows only my duties; date, time, room, course visible; PDF export works
US-T05-004	Admin	be clearly informed when a slot cannot be assigned to any invigilator and guided on how to resolve it	I can take corrective action before publishing the roster without missing any unassigned slot	Red banner and red slot highlighting appear after auto-assign with fully constrained pool; Publish button is disabled until all slots are resolved; all three resolution paths (expand pool, manual override, reschedule) are accessible from the panel
US-T06-001	Admin	schedule a thesis defense and detect conflicts with class routines	the defense can proceed without disrupting the normal class schedule	Defense scheduled; system warns if any committee member has a class conflict

Story ID	As a...	I want...	So that...	Acceptance Criteria
US-T07-001	Student	find which courses I still need after switching from Curriculum 2016 to 2023	I can plan my remaining years without missing or retaking courses	System displays missing courses, mapped equivalents, and remaining credits accurately
US-T08-001	Student	download a clean PDF of my weekly class routine	I can print it or use it offline without internet access	PDF downloads on click; timetable grid is complete and formatted for A4/Letter printing

6.2 Critical Use Cases

UC-01: Admin Generates and Publishes Class Routine

Preconditions: Admin is logged in; rooms, teachers, and curriculum are seeded in the system.

Main Flow:

1. Admin navigates to Class Routines and clicks Create New.
2. Selects academic year, semester, and section.
3. Adds courses and assigns teachers and rooms to time slots.
4. System performs real-time conflict detection on each assignment.
5. Admin reviews the conflict report and resolves all flagged conflicts.
6. Admin clicks Publish; system changes status to *published* and makes the routine visible in student and faculty portals.

Alternate Flow: If a conflict cannot be resolved immediately, admin saves as draft and returns later. If a teacher has no available slots, admin is warned and may choose a different teacher.

Postconditions: Class routine is published and accessible to all authenticated users; conflict report is empty.

UC-02: Admin Runs Workload-Balanced Invigilation Assignment

Preconditions: Exam routine is finalized; invigilation pool is defined; teacher unavailability constraints are recorded.

Main Flow:

1. Admin navigates to Invigilation Assignment and selects the exam routine.
2. Clicks Auto-Assign Invigilators.

3. System assigns teachers respecting all constraints (unavailability, teaching schedule, no double-assignment).
4. System distributes duties to minimize workload imbalance.
5. Admin reviews the generated roster and may swap individual assignments manually.

Alternate Flow — No Eligible Invigilator Available for a Slot: If no eligible invigilator remains for one or more slots after applying all constraints (unavailability, teaching overlap, double-booking), the system shall:

1. Display a persistent red banner at the top of the Invigilation Assignment Panel immediately after auto-assignment completes, reading: *"Warning: {N} exam slot(s) could not be assigned — no eligible invigilators available. Manual action required before publishing."*
2. Highlight each unassigned slot in red within the exam slot list on the right panel. Hovering the slot shows a tooltip: *"No eligible invigilator: all pool members are unavailable, teaching, or already assigned at this time."*
3. Move all unassigned slots into a dedicated **Unresolved Slots** section pinned to the top of the exam slot list, visually separated from successfully assigned slots.
4. Disable the Publish button; the button shall display the tooltip *"Resolve all unassigned slots before publishing"* on hover.
5. Offer the admin three inline resolution actions for each unassigned slot:
   (a) **Expand pool** — open the Invigilator Pool table (left panel) to add a previously excluded faculty member;
   (b) **Manual override** — assign any available faculty member via a dropdown on the unassigned slot row, bypassing pool restrictions; or
   (c) **Reschedule exam** — follow a direct navigation link to the Exam Routine editor to move the slot to a time when eligible invigilators are free.
6. Re-enable the Publish button and clear the red banner only after every slot has at least one assigned invigilator.

Postconditions: All exam slots have at least one invigilator; duty distribution is within 1 of the mean; roster is exportable.

UC-03: Student Uses Missing Course Finder After Curriculum Shift

Preconditions: Student is authenticated; curriculum equivalency mappings are defined by Admin; student has a record of completed courses.

Main Flow:

1. Student opens the Missing Course Finder and enters student ID, original curriculum version, and list of completed courses.
2. Selects target curriculum version and clicks Calculate.

3. System maps completed courses to equivalents in the new curriculum, identifies unmet requirements, and displays missing courses with total remaining credits.

Alternate Flow: If a completed course has no equivalency mapping, it is flagged as unmapped and excluded from the calculation; admin is prompted to add a mapping.

Postconditions: Student sees a complete list of remaining required courses and credits; report can be exported as PDF for advising use.

7. Data Requirements

7.1 Core Entities

Entity	Description	Key Fields	Constraints
User	Authenticated system user with role-based access	_id, googleId, email, name, role, isActive, createdAt	Email domain restricted; role in {student, teacher, admin, superadmin}
Curriculum	Versioned degree program course list	_id, version, program, totalCredits, status, courses[], createdBy	Version unique; status in {draft, published, archived}
Course	Individual course within a curriculum	code, title, credits, type, semester, prerequisites[]	Unique code per curriculum; type in {theory, lab, project}; semester 1–8
Room	Physical classroom or lab with capacity	_id, name, type, capacity, building, isActive	Capacity > 0; type in {classroom, lab, seminar}; name unique per building
ClassRoutine	Weekly class timetable per section/semester	_id, academicYear, semester, section, status, slots[]	Status in {draft, published}; slots conflict-free before publish
RoutineSlot	A single time-slot entry within a class routine	day, startTime, endTime, courseCode, teacherId, roomId	Day in Mon–Fri; startTime < endTime; all referenced entities must exist
ExamRoutine	Semester exam schedule with room and time assignments	_id, academicYear, semester, status, exams[]	Exams must not overlap for same section; rooms not over-capacity

Entity	Description	Key Fields	Constraints
ExamSlot	Individual exam event within an exam routine	courseCode, date, startTime, duration, rooms[], invigilators[]	Duration > 0; at least one room assigned
InvigilationDuty	Assignment record linking teacher to an exam slot	_id, examRoutineId, teacherId, examId, roomId, status	Status in {assigned, confirmed, swapped}; teacher not double-booked
Thesis	Student thesis/project record	_id, studentId, title, supervisorId, status, defenseDate, committeeMembers[]	Status in {registered, defended, completed}; studentId unique per active thesis
CurriculumMapping	Equivalency between courses across curriculum versions	_id, fromCurriculumId, toCurriculumId, fromCourseCode, toCourseCode	Must reference valid courses in both curricula

7.2 Data Dictionary

Field	Type	Req?	Validation / Constraint	Example
`email`	String	Yes	Must match `/@(cs|cse)\.du\.ac\.bd$/`; max 100 chars	`mithu@cse.du.ac.bd`
`role`	Enum String	Yes	One of: student, teacher, admin, superadmin	`admin`
`version`	String	Yes	Format: 4-digit year or year.minor; unique per program	`2023`
`code`	String	Yes	Format: 'CSE XXXX'; unique within curriculum	`CSE 4113`
`credits`	Number	Yes	Positive; typically 3.0 theory, 1.5 lab	`3`
`type`	Enum String	Yes	One of: theory, lab, project	`theory`
`semester`	Integer	Yes	Integer 1–8 inclusive	`7`

Field	Type	Req?	Validation / Constraint	Example
`day`	Enum String	Yes	One of: Monday, Tuesday, Wednesday, Thursday, Friday	`Monday`
`startTime`	String	Yes	HH:MM (24h); within 08:00–18:00; before endTime	`09:00`
`capacity`	Integer	Yes	Positive integer; maximum 500	`60`
`status`	Enum String	Yes	Entity-specific enum (see entity constraints)	`published`
`academicYe`	String	Yes	Format: YYYY-YYYY; end year = start + 1	`2024-2025`
`duration`	Integer	Yes	Exam duration in minutes; 30–240	`180`
`defenseDat`	Date	No	Must be a future date when scheduling; ISO 8601	`2026-05-15`
`grade`	String	No	Standard DU grade notation (A, A-, B+, ... F)	`A-`

7.3 Data Model

The system uses MongoDB (document-oriented NoSQL). Core relationships:

- **User (1) — manages — Curriculum (many):** Admin creates and edits curricula; `createdBy` field references User.
- **Curriculum (1) — contains — Course (many):** Each curriculum has an ordered list of course subdocuments; courses are embedded (not referenced) for atomic updates.
- **ClassRoutine (1) — contains — RoutineSlot (many):** Each slot embeds `teacherId` (ref: User) and `roomId` (ref: Room).
- **ExamRoutine (1) — contains — ExamSlot (many):** Each exam slot embeds `rooms[]` and `invigilators[]` as ObjectId references.
- **InvigilationDuty — joins — User and ExamSlot:** One duty record per teacher-exam assignment; enables efficient per-teacher and per-exam queries.
- **Thesis — references — User (multiple roles):** studentId, supervisorId, coSupervisorId, and committeeMembers[] are all ObjectId references to User.

- **CurriculumMapping — links — Curriculum + Course pairs:** fromCurriculumId.fromCourseCode maps to toCurriculumId.toCourseCode for the missing course finder.

Primary indexes: `User.email` (unique), `Curriculum.version` (unique per program), `ClassRoutine.(academicYear + semester + section)` (composite unique), `ExamRoutine.(academic + semester)` (composite unique).

8. UI and UX Requirements

8.1 Wireframes

The following wireframes describe the five primary user flows. Final high-fidelity designs will use React.js with Tailwind CSS.

WF-01: Landing Page / Login Screen

Purpose: Entry point for all users.

Layout: Centered card with University of Dhaka logo at top, system title "Academic Operations Suite", subtitle listing the five modules, and a prominent "Sign in with Google" button. Footer shows department name and academic year. Background: subtle DU blue gradient.

Key interactions: Clicking Sign in with Google initiates OAuth flow. On domain rejection, an error banner appears: "This system is restricted to `@cs.du.ac.bd` and `@cse.du.ac.bd` accounts."

WF-02: Admin Dashboard

Purpose: Admin home screen showing quick stats and navigation to all admin modules.

Layout: Top navigation bar with logo, user avatar, and logout button. Left sidebar with module links (Curriculum, Class Routines, Exam Routines, Invigilation, Thesis, Missing Course Finder, Users). Main content: 4 stat cards (Active Routines, Upcoming Exams, Pending Conflicts, Active Theses) plus Recent Activity feed and Quick Actions panel.

Key interactions: Clicking any stat card navigates to the relevant module. Conflict count card shows a red badge when conflicts > 0. Sidebar collapses to hamburger on smaller screens.

WF-03: Class Routine Builder

Purpose: Core admin interface for creating and editing a class routine with real-time conflict detection.

Layout: Full-width timetable grid (Monday–Friday columns, time slots as rows). Left panel: course and teacher assignment form. Right panel: live Conflict Report panel updated as slots are assigned. Toolbar: Save Draft, Check Conflicts, Publish. Color coding: Assigned (blue), Conflicted (red), Empty (light grey).

Key interactions: Conflicts detected within 1 second; conflicted cells turn red. Drag-and-drop between cells triggers re-check. Hovering a cell shows a tooltip with full slot details.

WF-04: Student Published Routine Portal

Purpose: Read-only view for students to see their published class and exam routines.

Layout: Filter bar: Year, Semester, Section dropdowns. Toggle between Grid View (weekly timetable) and List View (sortable table). Download PDF button top-right. Exam Routine tab shows a date-sorted list of upcoming exams.

Key interactions: Changing filters reloads the correct routine immediately. Clicking a grid cell shows a modal with full details. Download PDF generates a clean printable version.

WF-05: Invigilation Assignment Panel

Purpose: Admin interface to define the invigilation pool and run auto-assignment.

Layout: Split layout. Left: Invigilator Pool table (teacher name, duty count, availability badge). Right: Exam slot list with current assignments per slot; an **Unresolved Slots** section appears pinned at the top of the right panel when any slot cannot be assigned, highlighted in red. Toolbar: Auto-Assign, Filter by Date, Export Roster (PDF/Excel). Bottom: workload balance bar chart showing duty count per teacher.

Key interactions: Admin checks/unchecks teachers to include/exclude from pool. Auto-Assign populates right panel. If any slots are unresolvable, a persistent red banner appears at the top of the panel stating the number of unresolved slots; the Publish button is disabled until all slots are resolved. Each unresolved slot row shows three inline resolution actions: Expand Pool, Manual Override (dropdown), and Reschedule (link to Exam Routine editor). Individual assignments can be swapped by selecting a different eligible teacher. Export Roster generates the roster immediately.

8.2 UX and Accessibility Baseline

- **Clear navigation and labels:** All navigation items use plain language labels; icons accompanied by text.
- **Keyboard accessibility:** All core flows operable without a mouse; visible focus indicators.
- **Readable contrast and typography:** Minimum 4.5:1 contrast ratio for all body text; minimum 16 px base font size.
- **Responsive behavior:** Fully functional at 768 px (tablet) through 1920 px (desktop); mobile view optimized for read-only portal.
- **Error and feedback states:** All form validation errors shown inline with clear resolution guidance; success/failure toasts for all save/publish actions.
- **Loading states:** Spinner shown for all async operations >300 ms; skeleton screens for table loads.

9. Non-Functional Requirements

NFR ID format: `NFR-TXX-###`

NFR ID	Category	Requirement	Target / Metric
`NFR-T01-001`	Performance	All read API responses shall complete within 2 seconds under normal load (up to 100 concurrent users).	P95 < 2 s at 100 concurrent users in load test
`NFR-T01-002`	Performance	Conflict detection operations shall complete within 5 seconds for any routine size.	P95 < 5 s during conflict-heavy routine generation
`NFR-T02-001`	Security	All authentication via Google OAuth 2.0; passwords shall never be stored in any form.	Code review confirms no password storage; OAuth flow verified end-to-end
`NFR-T02-002`	Security	All data in transit shall be encrypted using HTTPS (TLS 1.2 or higher).	SSL Labs scan rates A or A+; no HTTP endpoints exposed
`NFR-T02-003`	Security	API endpoints shall enforce rate limiting of 100 requests per IP per minute.	Requests beyond limit return HTTP 429; verified with load tool
`NFR-T02-004`	Security	All user inputs shall be validated on frontend and backend; sanitized against XSS and injection.	OWASP ZAP scan shows no XSS or injection vulnerabilities
`NFR-T03-001`	Usability	The UI shall be fully functional and visually coherent on screen widths 768 px–1920 px.	All core flows tested on 768 px, 1280 px, and 1920 px without layout breaks
`NFR-T03-002`	Usability	All core features shall be accessible within 3 clicks from the dashboard.	Usability test with 3 users confirms no core feature requires more than 3 clicks

NFR ID	Category	Requirement	Target / Metric
`NFR-T04-001`	Reliability	System shall target 99% uptime during academic semester periods.	Uptime monitor reports ≥99% monthly over one full semester
`NFR-T04-002`	Reliability	MongoDB Atlas automated backups enabled; point-in-time recovery for the last 7 days.	Atlas backup config verified; successful restore from 3-day-old snapshot
`NFR-T05-001`	Maintainability	Backend shall follow modular architecture: controllers, services, routes, and models separated.	Code review confirms separation; no business logic in route files
`NFR-T05-002`	Maintainability	All REST API endpoints shall be documented in Swagger/OpenAPI 3.0 at `/api/docs`.	All endpoints reachable and documented; no 404 on any documented route
`NFR-T05-003`	Scalability	Backend architecture shall support horizontal scaling via containerization if needed.	Application containerized with Docker; load balanced instance test passes

10. Risks and Open Questions

10.1 Risk Register

Risk ID	Description	Probabilit	Impact	Mitigation
RISK-T01-001	Scope creep: too many features for one semester	High	High	Apply MoSCoW strictly; defer low-priority features; weekly scope review
RISK-T01-002	MongoDB Atlas free-tier storage/connection limits	Medium	Medium	Monitor Atlas metrics; use careful indexing; upgrade tier if quota approached
RISK-T01-003	Google OAuth domain restriction misconfiguration	Medium	High	Thorough testing of domain filter; manual admin override for edge cases
RISK-T01-004	Conflict detection algorithm complexity causing delays	Medium	Medium	Start with simple rule-based detection; optimize incrementally; set 5 s timeout fallback
RISK-T01-005	Team Formula1 billing API unavailability	Low	Low	Integration is optional; system fully functional without it; use adapter pattern
RISK-T01-006	Data loss during development	Low	High	Git version control from day one; Atlas backups enabled; no direct production DB edits
RISK-T01-007	Team member unavailability reducing velocity	Medium	Medium	Modular task ownership; knowledge sharing in weekly syncs; cross-training on critical modules

10.2 Open Questions

1. What is the exact structure and availability of the Team Formula1 billing API? *Owner: Team Newbie + Formula1 team; Target: Week 3*
2. Will the department provide a seed dataset of existing rooms, courses, and faculty for initial setup? *Owner: Course instructor; Target: Week 2*
3. How should the system handle partial curriculum equivalency (e.g., a 3-credit course maps to two 1.5-credit courses)? *Owner: Team Newbie + academic advisor; Target: Week 4*
4. What is the maximum number of concurrent users expected during peak periods (e.g., routine publication day)? *Owner: Dept. admin; Target: Week 2*
5. Are part-time or visiting faculty included in the invigilation pool by default? *Owner: Dept. admin; Target: Week 3*

11. Testability and Traceability

11.1 Requirement Traceability Matrix

Requirement ID	Story / Use Case	Planned Test ID
FR-T01-001	US-T01-002	TC-T01-001, TC-T01-002
FR-T01-002	US-T01-002	TC-T01-002
FR-T01-003	US-T01-001	TC-T01-003
FR-T01-005	US-T01-002	TC-T01-004, TC-T01-005
FR-T02-001	US-T02-001	TC-T02-001
FR-T02-002	US-T02-001	TC-T02-002
FR-T02-004	US-T02-002	TC-T02-003
FR-T03-001	US-T03-001	TC-T03-001
FR-T03-004	US-T03-001	TC-T03-002, TC-T03-003
FR-T03-006	US-T03-002	TC-T03-004
FR-T03-007	US-T03-003, US-T08-001	TC-T03-005
FR-T04-001	US-T04-001	TC-T04-001
FR-T04-003	US-T04-001	TC-T04-002
FR-T04-004	US-T04-002	TC-T04-003
FR-T05-002	US-T05-001	TC-T05-001
FR-T05-003	US-T05-002	TC-T05-002
FR-T05-005	US-T05-003	TC-T05-003
FR-T05-006	US-T05-004, UC-02	TC-T05-004, TC-T05-005
FR-T07-003	US-T07-001	TC-T07-001
FR-T08-001	US-T08-001	TC-T08-001
FR-T08-004	US-T08-001	TC-T08-002

11.2 Initial Manual Test Plan

Test ID	Scenario	Preconditions	Steps	Expected Result
`TC-T01-001`	Google Login – Valid Domain	App running; test account `@cse.du.ac.bd`	1. Open app 2. Click Sign in with Google 3. Authenticate with valid account	User logged in; redirected to dashboard; correct role shown
`TC-T01-002`	Google Login – Invalid Domain	App running; `@gmail.com` test account	1. Click Sign in with Google 2. Authenticate with `@gmail.com`	Login rejected; error message "Domain not authorized"; no session created
`TC-T01-003`	Super Admin Role Elevation	Logged in as Super Admin; target Teacher exists	1. Navigate to User Management 2. Select Teacher 3. Change role to Admin 4. Save	Role updated; target user sees Admin menus on next login
`TC-T01-004`	Unauthorized Route Access	Logged in as Student	1. Manually navigate to `/admin/routines/c`	HTTP 403 returned; redirected to error page; no admin content visible
`TC-T01-005`	JWT Token Expiry	Logged in; token manually expired	1. Log in 2. Expire token 3. Attempt authenticated action	Session expired; redirected to login; no API data returned
`TC-T02-001`	Create New Curriculum	Logged in as Admin	1. Go to Curriculum 2. Click Create 3. Enter version 2023, add 3 courses 4. Save	Curriculum saved in draft; appears in list with correct version and course count
`TC-T02-002`	Multi-Version Coexistence	Curriculum 2016 and 2023 both created	1. Open curriculum list 2. Verify both versions listed independently	Both versions visible; each shows its own independent course list
`TC-T02-003`	Curriculum Comparison	Two versions exist	1. Select Curriculum 2016 and 2023 2. Click Compare	Side-by-side view shows added (green), removed (red), modified (yellow)

Test ID	Scenario	Preconditions	Steps	Expected Result
TC-T03-001	Create Class Routine Draft	Admin logged in; rooms and courses seeded	1. Go to Class Routines 2. Create new for Year 4 Sem 1 3. Add 4 slots 4. Save	Routine saved in draft; all 4 slots visible in timetable grid
TC-T03-002	Room Conflict Detection	Room 301 assigned to Monday 09:00	1. Assign another course to Room 301 at Monday 09:00 2. Observe UI	Conflict indicator shown; slot highlighted red; conflict report updated
TC-T03-003	Teacher Conflict Detection	Teacher A assigned to Monday 09:00	1. Assign Teacher A to second Monday 09:00 slot	Teacher conflict flagged; system prevents saving conflicting assignment
TC-T03-004	Manual Drag-and-Drop	Routine draft with at least 2 slots	1. Drag slot from Monday 09:00 to Monday 11:00 2. Observe result	Slot moves; conflict check runs; if new conflict, slot highlighted red
TC-T03-005	Publish Class Routine	Draft routine with no conflicts	1. Open routine 2. Click Publish 3. Confirm	Status changes to published; routine visible in student portal
TC-T04-001	Create Exam Schedule	Admin logged in; courses and rooms available	1. Go to Exam Routines 2. Create for Semester 7 3. Assign 5 exams 4. Save	Exam schedule saved; all 5 exams listed with date, time, room
TC-T04-002	Exam Conflict Detection	Section A has exam at 09:00 Jan 15	1. Add another exam for Section A at 09:00 Jan 15 2. Attempt to save	System blocks save; error message indicating student group conflict
TC-T04-003	Export Exam Routine PDF	Exam routine published	1. Go to student portal 2. Open exam routine 3. Click Download PDF	PDF downloads; shows all exam entries with date, time, course, room

Test ID	Scenario	Preconditions	Steps	Expected Result
TC-T05-001	Teacher Declares Unavailability	Teacher logged in	1. Go to availability settings 2. Mark Jan 15 morning unavailable 3. Save	Unavailability saved; teacher not assigned to any Jan 15 morning invigilation
TC-T05-002	Auto-Assign Invigilators	Exam routine finalized; 10-teacher pool defined	1. Go to Invigilation 2. Click Auto-Assign 3. Review roster	Duties distributed; no teacher has more than 1 duty above the average
TC-T05-003	Export Individual Roster	Teacher A has 3 assigned duties	1. Filter roster by Teacher A 2. Click Export PDF	PDF shows only Teacher A's 3 duties with date, time, room, course
TC-T05-004	Unassignable Slot – Warning UI	Exam routine finalized; all pool members marked unavailable for Slot X	1. Go to Invigilation 2. Click Auto-Assign	Red banner appears: "Warning: 1 exam slot(s) could not be assigned"; Slot X highlighted red in Unresolved Slots section; Publish button disabled

Test ID	Scenario	Preconditions	Steps	Expected Result
TC-T05-005	Unassignable Slot – Resolution Paths	TC-T05-004 preconditions met; red banner visible	1. Hover Slot X row – verify tooltip text 2. Click Expand Pool – verify Pool table opens 3. Click Manual Override dropdown – verify any faculty selectable 4. Click Reschedule link – verify navigation to Exam Routine editor 5. Assign invigilator via Manual Override 6. Verify Publish button re-enables and red banner disappears	Tooltip shows correct constraint explanation; all three resolution paths functional; after manual assignment Publish button becomes active and banner clears
TC-T07-001	Missing Course Calculation	Curriculum mapping defined; student completed 20 courses	1. Open Missing Course Finder 2. Enter student details and completed courses 3. Select target Curriculum 2023 4. Calculate	Correct missing course list and remaining credit total matching manual calculation
TC-T08-001	Student Views Class Routine	Student logged in; Year 4 Sem 1 Sec A published	1. Log in as student 2. Select Year 4, Sem 1, Sec A 3. View routine	Correct timetable grid displayed; all slots show course, teacher, room, time
TC-T08-002	Download Routine PDF	Routine published; student logged in	1. Open published routine 2. Click Download PDF	PDF downloads; timetable grid printed cleanly with all slot details

Appendix A: MoSCoW Feature Prioritization

Feature	Priority
Google OAuth + Role Management	**Must Have**
Curriculum Management (CRUD + Versioning)	**Must Have**
Class Routine Generation with Conflict Detection	**Must Have**
Exam Routine Generation	**Must Have**
Invigilation Assignment	**Must Have**
Published Student / Faculty Portal	**Must Have**
Missing Course Finder	**Should Have**
Thesis Supervision and Defense Scheduling	**Should Have**
PDF / Excel Export	**Should Have**
Curriculum Comparison View	**Should Have**
Billing System Integration (Team Formula1)	**Could Have**
Email Notifications to Invigilators	**Could Have**
Dark Mode	Won't Have (this version)
Mobile App	Won't Have (this version)

Appendix B: API Endpoint Summary

Method	Endpoint	Auth Role	Description
GET	`/api/v1/auth/google`	Public	Initiate Google OAuth flow
GET	`/api/v1/auth/callback`	Public	OAuth callback handler
POST	`/api/v1/auth/logout`	Any	Logout and invalidate JWT
GET	`/api/v1/users`	Admin	List all users with roles
PUT	`/api/v1/users/:id/role`	Super Admin	Update user role
GET	`/api/v1/curricula`	Any	List all curriculum versions
POST	`/api/v1/curricula`	Admin	Create new curriculum version
GET	`/api/v1/curricula/:id`	Any	Get curriculum details with courses
PUT	`/api/v1/curricula/:id`	Admin	Edit curriculum (auto-versions if published)
GET	`/api/v1/routines/class`	Any	List class routines
POST	`/api/v1/routines/class`	Admin	Create class routine draft
POST	`/api/v1/routines/class/che`	Admin	Run conflict detection
PUT	`/api/v1/routines/class/:id/publish`	Admin	Publish class routine
GET	`/api/v1/routines/exam`	Any	List exam routines
POST	`/api/v1/routines/exam`	Admin	Create exam routine
GET	`/api/v1/invigilation`	Admin	List all invigilation duties
POST	`/api/v1/invigilation/assign`	Admin	Auto-assign invigilators with balancing
GET	`/api/v1/thesis`	Admin/Teacher	List thesis records
POST	`/api/v1/thesis`	Admin	Register new thesis
POST	`/api/v1/missing-courses`	Any Auth	Calculate missing courses for a student

Method	Endpoint	Auth Role	Description
GET	`/api/v1/rooms`	Any	List available rooms and labs
POST	`/api/v1/rooms`	Admin	Create room or lab
GET	`/api/v1/export/routine/:id`	Any Auth	Export routine as PDF or Excel

Appendix C: References

1. IEEE Std 830-1998: *IEEE Recommended Practice for Software Requirements Specifications*
2. Routine reference portal: https://routine.farefin.com/
3. Curriculum shift example: https://docs.google.com/spreadsheets/d/1lX9IrMGa9XMRvncLe_Kd-c_v5PORr8GQT1wYOV1eT_Y
4. Google OAuth 2.0 Documentation: https://developers.google.com/identity/protocols/oauth2
5. PostgreSQL Documentation: https://www.postgresql.com
6. React.js Documentation: https://reactjs.org/docs/
7. Supabase Documentation: https://supabase.com/